

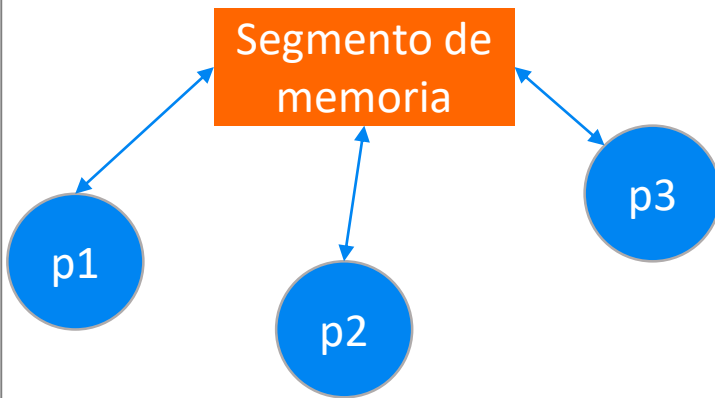
COMUNICACIÓN ENTRE PROCESOS



Comunicación entre procesos (Interprocess Communication - IPC)

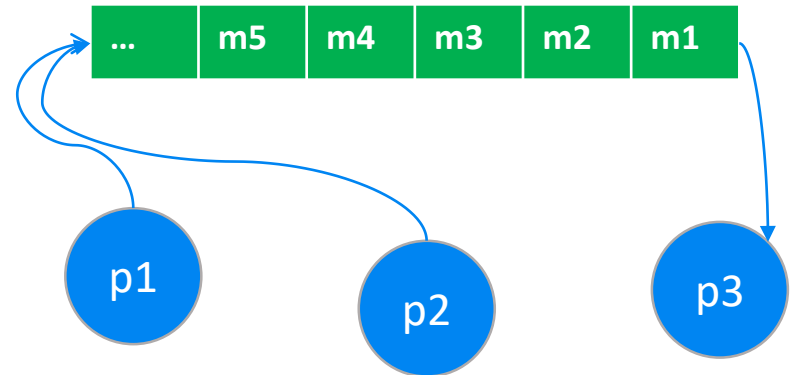
Memoria Compartida

SEGMENTO DE MEMORIA COMPARTIDA

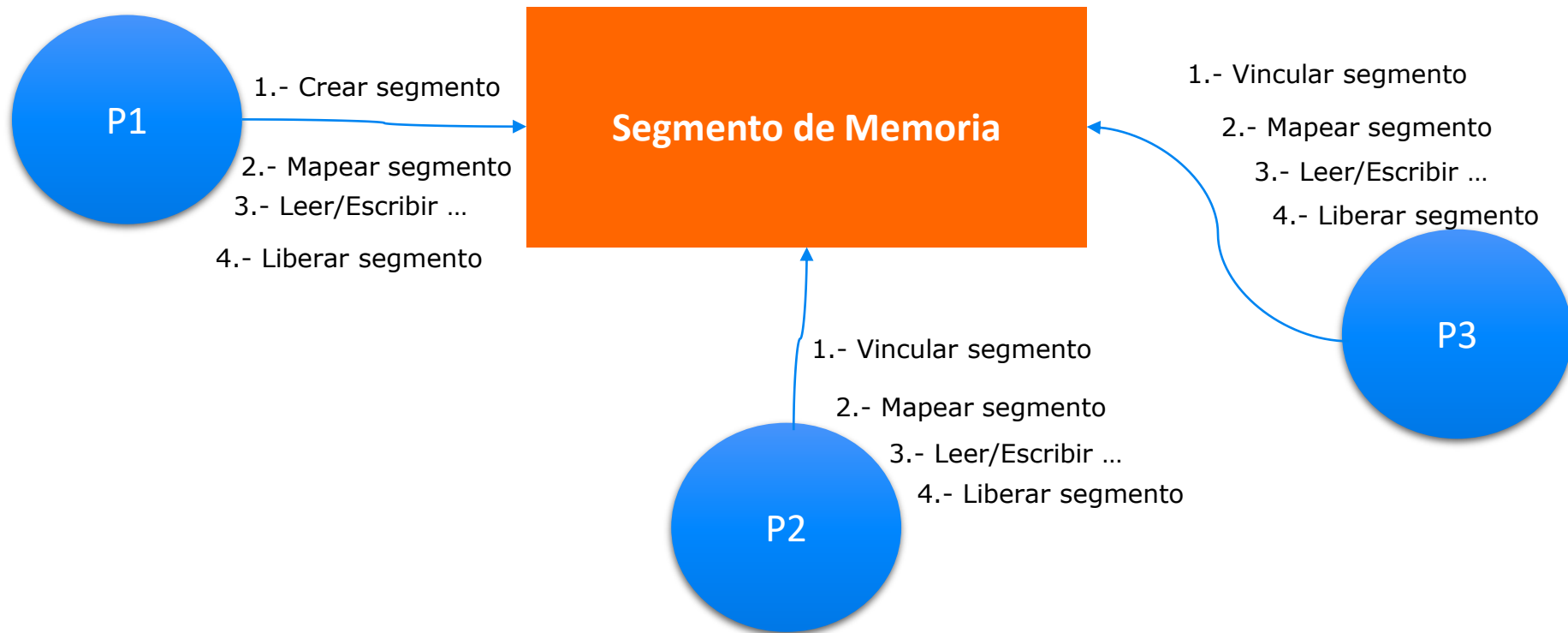


Mensajes

COLA DE MENSAJES



IPC: Segmento de Memoria Compartida



IPC: Segmento de Memoria Compartida

1. Creación de un nuevo segmento de memoria compartida o acceder a uno existente.
 - Llamada al sistema: **shmget (operación 1)**
2. Mapeado del segmento de memoria compartida al espacio de direcciones del proceso.
 - Llamada al sistema: **shmat (operación 2)**
3. Lectura o escritura. En esta zona se lee y se escribe como en cualquier dirección de memoria del proceso. Por ser una zona de memoria compartida, hay que tener cuidado cuando se la utiliza.
4. Desenlace del segmento de memoria compartida por parte del proceso.
 - Llamada al sistema: **shmdt**



IPC: Segmento de Memoria Compartida

1.- Crear u obtener un nuevo segmento de memoria

int shmget (key_t key, size_t size, int shmflg)

Por ejemplo:

```
struct info{  
    char mensaje[255];    /* dato del segmento */  
};  
  
#define KEY  ((key_t) (1243))    /* número de llave */  
#define SEGSIZE  sizeof (struct info)  /* longitud del segmento */
```

....

id = shmget(KEY, SEGSIZE, IPC_CREAT | 0666);



**Crea un
segmento**

id = shmget(KEY, SEGSIZE, 0);



**Vincula a un
segmento**

Definiciones

IPC: Segmento de Memoria Compartida

2.- Mapear un segmento de memoria

char *shmat(int shmid, void *shmaddr, int shmflg);

Por ejemplo:

...

```
ctrl = (struct info*) shmat(id,0,0);
```

4.- Desvincular un segmento de memoria

int shmdt(void *shmaddr)

Por ejemplo:

...

```
resul = shmdt(ctrl);
```

IPC: Segmento de memoria: Ejemplo

Proceso 1

```
struct info *ctrl;
// obtener un identificador
id = shmget(KEY, SEGSIZE, 0);
if (id < 0){
    printf(" FALLO EL shmget \n");
}
//asociar la memoria a la estructura
ctrl = (struct info*) shmat(id,0,0);
if (ctrl <= (struct info *) (0)){
    printf(" Error en el shmat \n"); exit(2);
}
// operar sobre memoria compartida
if ( argc == 1 )
    strcpy( ctrl->mensaje, "\0" );
else
    strcpy( ctrl->mensaje, argv[1]);
```

Proceso 2

```
struct info{
    char mensaje[255];
};

#define KEY  ((key_t) (1243))
#define SEGSIZE  sizeof (struct info)
//tamaño de la estructura

if (ctrl <= (struct info *) (0)){
    printf(" Error fallo shmat \n"); exit(2);
}
//valor inicial al segmento de memoria
compartida
strcpy(ctrl->mensaje, "Sistemas
Operativos");
```

IPC: Segmento de memoria: Ejemplo

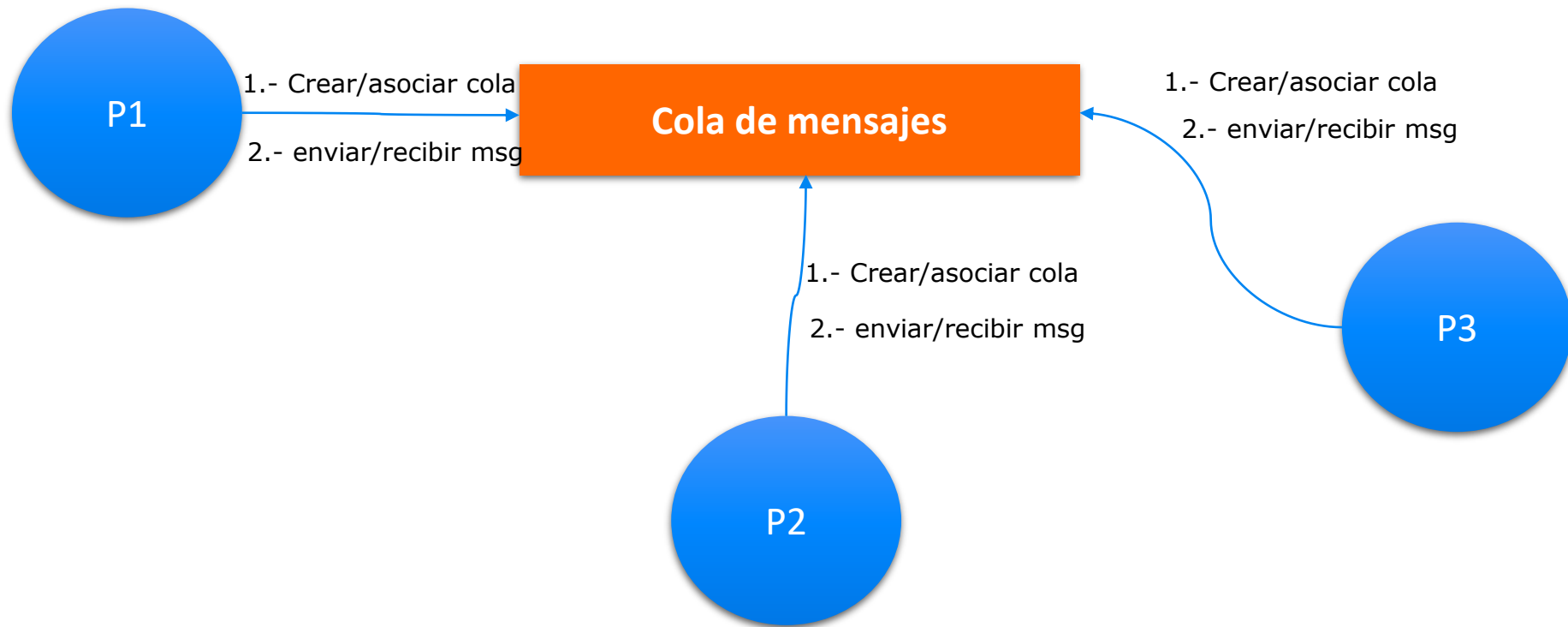
Proceso 1

```
struct info *ctrl;
// obtener un identificador
id = shmget(KEY, SEGSIZE, 0);
if (id < 0){
    printf(" FALLO EL shmget \n");  exit(1);
}
//asociar la memoria a la estructura local
ctrl = (struct info*) shmat(id,0,0);
if (ctrl <= (struct info *) (0)){
    printf(" Error en el shmat \n");  exit(2);
}
// operar sobre memoria compartida
if ( argc == 1 )
    strcpy( ctrl->mensaje, "\0" );
else
    strcpy( ctrl->mensaje, argv[1]);
```

Proceso 2

```
struct info *ctrl;
// crear la memoria compartida
id = shmget(KEY, SEGSIZE, IPC_CREAT |
0666);
if (id < 0){
    printf("Error fallo el shmget \n");  exit(1);
}
// asociar la memoria a la estructura ctrl
ctrl = (struct info*) shmat(id,0,0);
if (ctrl <= (struct info *) (0)){
    printf(" Error fallo shmat \n");  exit(2);
}
//valor inicial al segmento de memoria
compartida
strcpy(ctrl->mensaje, "Sistemas
Operativos");
```

IPC: Colas de Mensajes



IPC: Colas de Mensajes

1. Creación/asociación a una cola de mensajes.
 - Llamada al sistema: **msgget (operación 1)**
2. Lectura o escritura
 - Llamada al sistema: **msgsnd, msgrcv (operación 2)**

IPC: Colas de Mensajes

1.- Crear o asociar cola de mensajes

int msgget(key_t key, int msgflg)

Por ejemplo:

```
queueID = msgget(key, IPC_CREAT|0666);
```

```
queueID = msgget(key, 0666);
```

Crea y asocia

asocia

Estructura del mensaje

```
struct mensaje{  
    long tipo;  
    int dato_1;  
    ...  
};
```

En todas las
estructuras

IPC: Colas de Mensajes

2.- Envío y recepción de mensajes.

int msgsnd(int msqid, struct msgbuf *msgp, int msgsz, int msgflg);

int msgrcv(int msqid, struct msgbuf *msgp, int msgsz, long msgtyp, int msgflg);

- Si msgtyp = 0 - se accederá al primer mensaje que se encuentre en la cola independientemente de su tipo.
- Si msgtyp > 0 - se accederá al primer mensaje del tipo **msgtyp** que se encuentre almacenado en la cola.
- Si msgtyp < 0 - se accederá al primer mensaje cuyo tipo sea menor o igual al valor absoluto de **msgtyp** y a la vez sea el menor de todos.

IPC: Colas de Mensajes

Ejemplo:

```
...
longitud = sizeof(struct mensaje) - sizeof(long);
if (msgsnd(idmsg, &mimensaje, longitud, 0) == -1) {
    fprintf(stderr, ``Error en la escritura del mensaje\n");
    exit(1);
}
...

if (msgrcv(idmsg, &mimensaje, longitud, 1, 0) == -1) {
    fprintf(stderr, ``Error en la escritura del mensaje\n");
    exit(1);
}
```

IPC: Colas de mensajes

FORMATO DEL MENSAJE

```
struct mensaje{  
    long tipo;  
    int dato_1;  
    int dato_2;  
};
```

Primer campo del mensaje.
Define el tipo de mensaje

IPC: Colas de Mensajes: Ejemplo

Emisor

```
.... inicialización de la cola....  
idmsg = msgget(key, IPC_CREAT|0666);  
mimensaje.tipo = 1;  
mimensaje.dato_1 = 24;  
mimensaje.dato_2 = 4;  
  
longitud = sizeof(struct mensaje) -  
sizeof(long);  
  
if (msgsnd(idmsg, &mimensaje, longitud,  
0) == -1)  
{  
    printf("Error en la escritura mensaje\n");  
    exit(1); }  
}
```

Receptor

```
.... vinculación a la cola....  
idmsg = msgget(key, 0666);  
  
longitud = sizeof(struct mensaje) -  
sizeof(long);  
  
if (msgrcv(idmsg, &mimensaje, longitud,  
0, 0) == -1)  
{  
    printf("Error en la lectura mensaje\n");  
    exit(1); }  
}
```